SCHOOL OF MATHEMATICAL SCIENCE

Tribhuvan University

Lab Report of

Artificial Intelligence
(BDS 251)

Bachelor in Data Science
Institute of Science and Technology

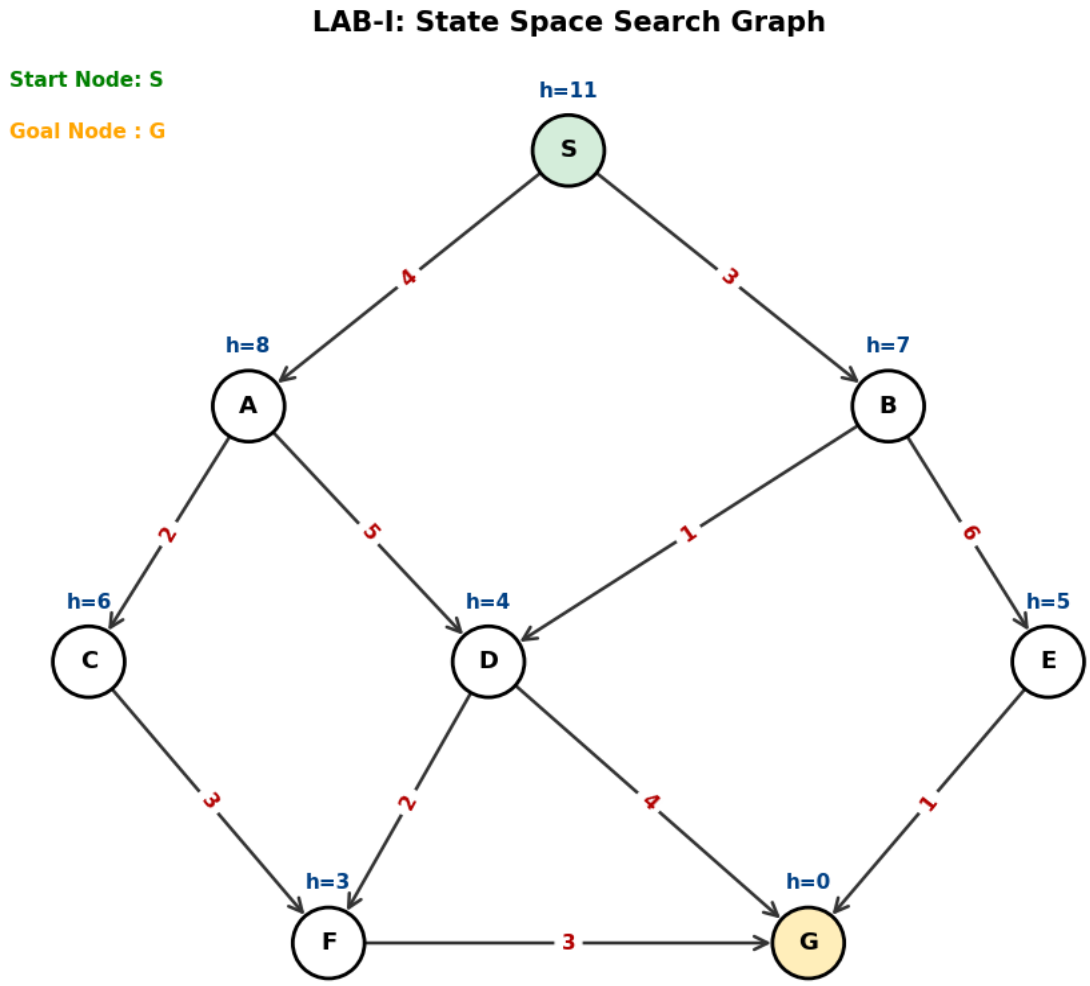
Submitted by:
Name: Prasanna Pokharel
Roll no: 24

Submitted to:
Mr. Arjun. K. Lamichhane

Contents

LAB-I:

GRAPH:




```
graph = {
'S': [('A', 4), ('B', 3)],
'A': [('C', 2), ('D', 5)],
'B': [('D', 1), ('E', 6)],
'C': [('F', 3)],
'D': [('F', 2), ('G', 4)],
'E': [('G', 1)],
'F': [('G', 3)],
'G': []
}

# Heuristic values h(n) estimating distance to Goal 'G'
heuristics = {'S': 11,'A': 8,'B': 7,'C': 6,'D': 4,'E': 5,'F': 3,'G': 0}
start_node = 'S'
goal_node = 'G'
```

1. WAP to implement BFS algorithm.

```
from collections import deque
from common_data import graph, start_node, goal_node, show_student_info

def bfs(graph, start, goal):
    queue = deque([[start]])
    visited = {start}

    while queue:
        path = queue.popleft()
        node = path[-1]

        if node == goal:
            return path

        for neighbor, _ in graph.get(node, []):
            if neighbor not in visited:
                visited.add(neighbor)
                queue.append(path + [neighbor])
    return None

path = bfs(graph, start_node, goal_node)
print(f"BFS Path from {start_node} to {goal_node}: {path}")
show_student_info("1 - BFS")
```

Output:

```
BFS Path from S to G: ['S', 'A', 'D', 'G']

====================================
Name         : Prasanna Pokharel
Roll No      : 24
Lab Number   : Lab I
Task         : 1 - BFS
====================================
```

2. WAP to implement DFS algorithm.

```
from common_data import graph, start_node, goal_node, show_student_info

def dfs(graph, node, goal, visited=None, path=None):
    if visited is None:
        visited = set()
    if path is None:
        path = []

    visited.add(node)
```

```
    path.append(node)

    if node == goal:
        return path

    for neighbor, _ in graph.get(node, []):
        if neighbor not in visited:
            res = dfs(graph, neighbor, goal, visited, path)
            if res:
                return res

    path.pop()
    return None

path = dfs(graph, start_node, goal_node)
print(f"DFS Path from {start_node} to {goal_node}: {path}")
show_student_info("2 - DFS")
```

OUTPUT:

```
DFS Path from S to G: ['S', 'A', 'C', 'F', 'G']

====================================
Name         : Prasanna Pokharel
Roll No      : 24
Lab Number   : Lab I
Task         : 2 - DFS
====================================
```

3. WAP to implement UCS algorithm.

```
import heapq
from common_data import graph, start_node, goal_node, show_student_info

def ucs(graph, start, goal):
    pq = [(0, start, [start])]
    visited = {}

    while pq:
        cost, node, path = heapq.heappop(pq)

        if node == goal:
            return cost, path

        if node in visited and visited[node] <= cost:
            continue
        visited[node] = cost

        for neighbor, weight in graph.get(node, []):
            if neighbor not in visited:
                heapq.heappush(pq, (cost + weight, neighbor, path + [neighbor]))
```

```
    return float('inf'), []

cost, path = ucs(graph, start_node, goal_node)
print(f"UCS Optimal Path: {path} with Total Cost: {cost}")
show_student_info("3 - UCS")
```

OUTPUT

```
UCS Optimal Path: ['S', 'B', 'D', 'G'] with Total Cost: 8

========================================
Name         : Prasanna Pokharel
Roll No      : 24
Lab Number   : Lab I
Task         : 3 - UCS
========================================
```

4. WAP to implement DLS algorithm.

```
from common_data import graph, start_node, goal_node, show_student_info

def dls(graph, node, goal, limit, path):
    path.append(node)
    if node == goal:
        return True
    if limit <= 0:
        path.pop()
        return False

    for neighbor, _ in graph.get(node, []):
        if neighbor not in path:
            if dls(graph, neighbor, goal, limit - 1, path):
                return True

    path.pop()
    return False

limit = 4
path = []
found = dls(graph, start_node, goal_node, limit, path)

print(f"Goal {goal_node} reached within limit {limit}: {path if found else 'Not Found'}")
show_student_info("4 - DLS")
```

OUTPUT:

```
Goal G reached within limit 4: ['S', 'A', 'C', 'F', 'G']

==================================
Name         : Prasanna Pokharel
Roll No      : 24
Lab Number   : Lab I
Task         : 4 - DLS
==================================
```

5. WAP to implement A* search algorithm.

```
import heapq
from common_data import graph, heuristics, start_node, goal_node, show_student_info

def a_star(graph, h, start, goal):
    pq = [(h[start], 0, start, [start])]
    g_cost = {start: 0}

    while pq:
        f, g, node, path = heapq.heappop(pq)

        if node == goal:
            return g, path

        for neighbor, weight in graph.get(node, []):
            tentative_g = g + weight
            if neighbor not in g_cost or tentative_g < g_cost[neighbor]:
                g_cost[neighbor] = tentative_g
                f_cost = tentative_g + h.get(neighbor, 0)
                heapq.heappush(pq, (f_cost, tentative_g, neighbor, path +
[neighbor]))
    return float('inf'), []

cost, path = a_star(graph, heuristics, start_node, goal_node)
print(f"A* Path: {path} with Cost: {cost}")
show_student_info("5 - A*")
```

OUTPUT:

```
A* Path: ['S', 'B', 'D', 'G'] with Cost: 8

========================================
Name         : Prasanna Pokharel
Roll No      : 24
Lab Number   : Lab I
Task         : 5 - A*
========================================
```

6. WAP to implement General Hill climbing search algorithm.

```
from common_data import graph, heuristics, start_node, show_student_info

def hill_climbing(graph, h, start):
    current = start
    path = [current]

    while True:
```

```
        neighbors = graph.get(current, [])
        if not neighbors:
            break

        # Select neighbor with the lowest heuristic value (closest to goal)
        best_neighbor = min(neighbors, key=lambda n: h[n[0]])[0]

        # Stop if no strictly better neighbor exists (local optimum reached)
        if h[best_neighbor] >= h[current]:
            break

        current = best_neighbor
        path.append(current)

    return path, current, h[current]

path, peak, h_val = hill_climbing(graph, heuristics, start_node)
print(f"Hill Climbing Path: {path}")
print(f"Terminated at Node: {peak} with Heuristic: {h_val}")
show_student_info("6 - HC")
```

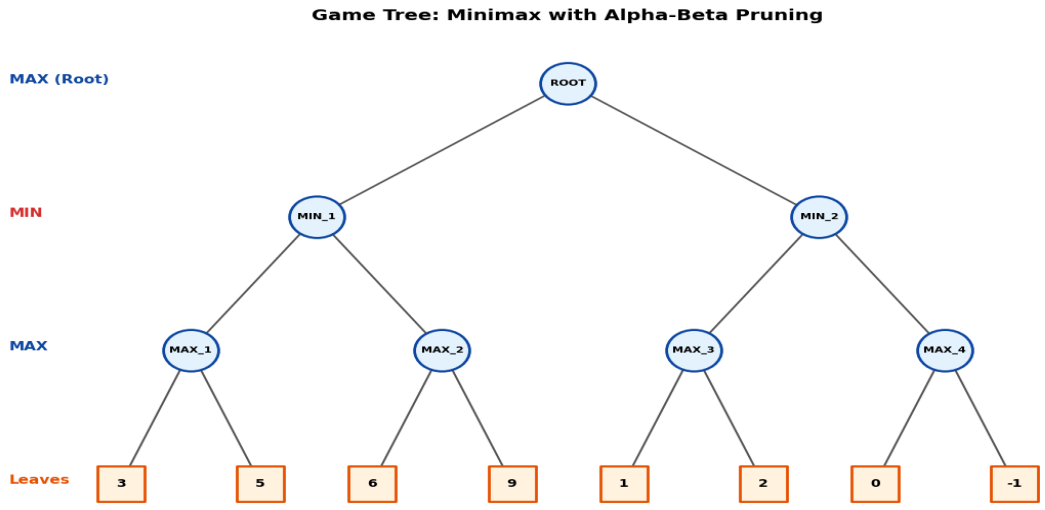
OUTPUT:

```
Hill Climbing Path: ['S', 'B', 'D', 'G']
Terminated at Node: G with Heuristic: 0

========================================
Name         : Prasanna Pokharel
Roll No      : 24
Lab Number   : Lab I
Task         : 6 - HC
========================================
```

7. WAP to perform minimax algorithm with alpha-beta pruning

Graph:



```python
import math
from common_data import game_tree, leaf_values, show_student_info

def alpha_beta(node, is_max, alpha, beta):
    # Terminal leaf check
    if node in leaf_values:
        return leaf_values[node]

    children = game_tree.get(node, [])

    if is_max:
        max_eval = -math.inf
        for child in children:
            eval_score = alpha_beta(child, False, alpha, beta)
            max_eval = max(max_eval, eval_score)
            alpha = max(alpha, eval_score)
            if beta <= alpha:
                print(f"Pruned remaining children of {node} (Beta={beta} <=
Alpha={alpha})")
                break
        return max_eval
    else:
        min_eval = math.inf
        for child in children:
            eval_score = alpha_beta(child, True, alpha, beta)
            min_eval = min(min_eval, eval_score)
            beta = min(beta, eval_score)
            if beta <= alpha:
                print(f"Pruned remaining children of {node} (Beta={beta} <=
Alpha={alpha})")
                break
        return min_eval

if __name__ == "__main__":
    initial_alpha = -math.inf
    initial_beta = math.inf

    root_val = alpha_beta('ROOT', is_max=True, alpha=initial_alpha,
beta=initial_beta)

    print(f"\nOptimal Root Evaluation Score: {root_val}")
    show_student_info("07 - Minimax with Alpha-Beta Pruning")
```

OUTPUT:

```
Pruned remaining children of MAX_2 (Beta=5 <= Alpha=6)
Pruned remaining children of MIN_2 (Beta=2 <= Alpha=5)

Optimal Root Evaluation Score: 5

========================================
Name         : Prasanna Pokharel
Roll No      : 24
Lab Number   : Lab I
Task         : 07 - Minimax with Alpha-Beta Pruning
========================================
```

LAB-II(Prolog)

1. What is prolog? Describe its history and current uses in brief.

Prolog (short for *PROgrammation en LOGique*) is a high-level declarative programming language based on formal logic—specifically first-order predicate calculus. Unlike imperative languages that require step-by-step algorithms, Prolog programs define relations through **facts** and **rules**, allowing the system's inference engine to deduce answers via resolution and backtracking.

- **History:** Developed around 1972 by **Alain Colmerauer** and Philippe Roussel in Marseille, France, with theoretical collaboration from Robert Kowalski at the University of Edinburgh. It gained global prominence in the 1980s as the core technology of Japan's Fifth Generation Computer Systems project.
- **Current Uses:**
  - **Natural Language Processing (NLP):** Parsing grammars using Definite Clause Grammars (DCGs).
  - **Expert Systems & Rule Engines:** Automated reasoning in legal, medical, and financial advisory software.
  - **Constraint Logic Programming (CLP):** Complex scheduling, planning, and combinatorial optimization problems.
  - **Semantic Web & Knowledge Graphs:** Querying linked data and ontologies.
  -

2. Write a program to display your name in prolog.

Prolog:

```
print_name :-
    write('Prasanna Pokharel').


display_info :-
    write('Student Name: [Prasanna Pokharel]'), nl,
    write('Roll No: [24]'), nl,
    write('Lab Number: Lab II-1'), nl.
```

Output:

```
31 ?- print_name.
Prasanna Pokharel
true.

32 ?- display_info.
Student Name: [Prasanna Pokharel]
Roll No: [24]
Lab Number: Lab II-1
true.
```

3. Write a prolog program to represent few basic facts and perform queries (eg elephant is an animal. Elephant is bigger than horse etc.)

Prolog:

```
% Facts

animal(giraffe).
animal(horse).
animal(dog).

bigger(giraffe, horse).
bigger(horse, dog).

% Print verification helper
print_footer :-
    nl,
    write('------------------------------'), nl,
    write('Name: [Prasanna Pokharel] | Roll: [24] | Lab: Lab II-2'), nl.
```

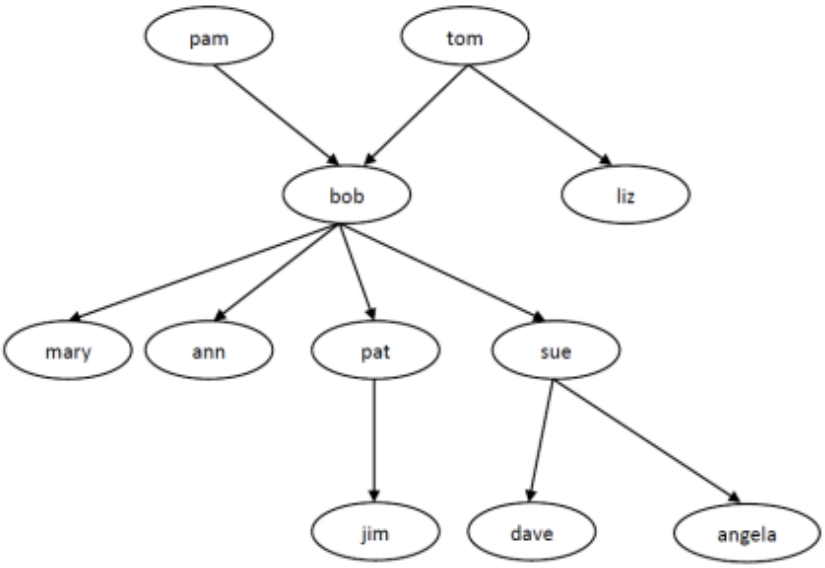
Output:

```
35 ?- bigger(dog, giraffe), print_footer.
false.

36 ?- print_footer.

------------------------------
Name: [Prasanna Pokharel] | Roll: [24] | Lab: Lab II-2
true.
```

4. For the above family tree:



a. Represent all members of family with their gender as property.

```
% (a) Gender specifications
male(tom).
male(bob).
male(pat).
male(jim).
male(dave).

female(pam).
female(liz).
female(mary).
female(ann).
female(sue).
female(angela).
```

b. Represent all parent relationship of the family.(i.e. all fact of form parent(X,Y))

```
% (b) Parent relationships: parent(Parent, Child)
parent(pam, bob).
parent(tom, bob).
parent(tom, liz).

parent(bob, mary).
```

```
parent(bob, ann).
parent(bob, pat).
parent(bob, sue).

parent(pat, jim).

parent(sue, dave).
parent(sue, angela).
```

c. Is tom parent of jim? No.

```
38 ?- parent(tom,jim)
|    .
false.

39 ?- print_footer.

------------------------------
Name: [Prasanna Pokharel] | Roll: [24] | Lab: Lab II-5
true.
```

d. Is angela male? No.

```
40 ?- male(angela).
false.

41 ?- print_footer.

------------------------------
Name: [Prasanna Pokharel] | Roll: [24] | Lab: Lab II-5
true.
```

e. Is bob parent of pat? Yes.

```
42 ?- parent(bob,pat)
|    .
true.

43 ?- print_footer.

------------------------------
Name: [Prasanna Pokharel] | Roll: [24] | Lab: Lab II-5
true.
```

f. Is liz parent of pat? No.

```
44 ?- parent(liz,pat)
|    .
false.

45 ?- print_footer.

------------------------------
Name: [Prasanna Pokharel] | Roll: [24] | Lab: Lab II-5
true.
```

5. For above family tree list following:

Prolog:

```
% Rules
grandparent(GP, GC) :-
    parent(GP, P),
    parent(P, GC).

granddaughter(GD, GP) :-
    female(GD),
    grandparent(GP, GD).

common_parent(X, Y) :-
    parent(P, X),
    parent(P, Y),
    X \= Y.

sibling(X, Y) :-
    parent(P, X),
    parent(P, Y),
    X \= Y.

sister(Sister, Person) :-
    female(Sister),
    parent(P, Sister),
    parent(P, Person),
    Sister \= Person.

uncle(Uncle, NieceNephew) :-
    male(Uncle),
    parent(P, NieceNephew),
    sibling(Uncle, P).
```

a) List all male family members

```
51 ?- male(X).
X = tom ;
X = bob ;
X = pat ;
X = jim ;
X = dave.

51 ?- print_footer.

------------------------------
Name: [Prasanna Pokharel] | Roll: [24] | Lab: Lab II-5
true.
```

b) List all female family members.

```
52 ?- female(X).
X = pam ;
X = liz ;
X = mary ;
X = ann ;
X = sue ;
X = angela.

53 ?- print_footer.

------------------------------
Name: [Prasanna Pokharel] | Roll: [24] | Lab: Lab II-5
true.
```

c) Who is liz's parent?

```
54 ?- parent(P,liz).
P = tom.

55 ?- print_footer.

------------------------------
Name: [Prasanna Pokharel] | Roll: [24] | Lab: Lab II-5
true.
```

d) Who are bob's children?

```
56 ?- parent(bob,C).
C = mary ;
C = ann ;
C = pat ;
C = sue.

57 ?- print_footer.

------------------------------
Name: [Prasanna Pokharel] | Roll: [24] | Lab: Lab II-5
true.
```

e) Find all parent children relationship.(find all X and Y such that X is parent of Y).

```
58 ?- parent(P,C).
P = pam,
C = bob ;
P = tom,
C = bob ;
P = tom,
C = liz ;
P = bob,
C = mary ;
P = bob,
C = ann ;
P = bob,
C = pat ;
P = bob,
C = sue ;
P = pat,
C = jim ;
P = sue,
C = dave ;
P = sue,
C = angela.

59 ?- print_footer.

------------------------------
Name: [Prasanna Pokharel] | Roll: [24] | Lab: Lab II-5
true.
```

f) Who is grand parent of jim?

```
61 ?- grandparent(G,jim).
G = bob .

62 ?- print_footer.

------------------------------
Name: [Prasanna Pokharel] | Roll: [24] | Lab: Lab II-5
true.
```

g) Who are tom's granddaughters?

```
63 ?- grandparent(tom,Y),female(Y).
Y = mary ;
Y = ann ;
Y = sue ;
false.

64 ?- print_footer.

------------------------------
Name: [Prasanna Pokharel] | Roll: [24] | Lab: Lab II-5
true.
```

h) Does ann and pat have common parent?

Yes, ann and pat have common parent.

```
65 ?- parent(P,ann),parent(P,pat).
P = bob.

66 ?- print_footer.

------------------------------
Name: [Prasanna Pokharel] | Roll: [24] | Lab: Lab II-5
true.
```

i) Who are the siblings of ann?

```
67 ?- sibling(X,ann).
X = mary ;
X = pat ;
X = sue ;
false.

68 ?- print_footer.

------------------------------
Name: [Prasanna Pokharel] | Roll: [24] | Lab: Lab II-5
true.
```

j) List all the sisters of pat.

```
70 ?- sister(S,pat).
S = mary ;
S = ann ;
S = sue ;
false.

71 ?- print_footer.

------------------------------
Name: [Prasanna Pokharel] | Roll: [24] | Lab: Lab II-5
true.
```

k) List all the uncles of Angela.

Angela has no uncles.

```
72 ?- uncle(U,angela).
U = pat ;
false.

73 ?- print_footer.

------------------------------
Name: [Prasanna Pokharel] | Roll: [24] | Lab: Lab II-5
true.
```

6. Write a program to display factorial of a user defined number using recursion.

Prolog:

```
% Base case: 0! = 1
factorial(0, 1).

% Recursive step
factorial(N, Result) :-
    N > 0,
    N1 is N - 1,
    factorial(N1, SubResult),
    Result is N * SubResult.

% Interactive caller displaying student info
calculate_factorial :-
    write('Enter a number: '),
    read(N),
    factorial(N, Ans),
```

```
    format('Factorial of ~w is ~w.~n', [N, Ans]),
    write('-----------------------------'), nl,
    write('Name: [Prasanna Pokharel] | Roll: [24] | Lab: Lab II-6'), nl.
```

OUTPUT:

```
75 ?- calculate_factorial.
Enter a number: 7
|: .
Factorial of 7 is 5040.
-----------------------------
Name: [Prasanna Pokharel] | Roll: [24] | Lab: Lab II-6
true .
```

7. Write a program to display Fibonacci sequence up to user defined number using recursion.

Prolog:

```
% fib(Index, Value)
fib(0, 0).
fib(1, 1).
fib(N, Val) :-
    N > 1,
    N1 is N - 1,
    N2 is N - 2,
    fib(N1, V1),
    fib(N2, V2),
    Val is V1 + V2.

% Helper to iterate from 0 to N
generate_fib(Current, Limit) :-
    Current =< Limit,
    fib(Current, Val),
    format('~w ', [Val]),
    Next is Current + 1,
    generate_fib(Next, Limit).
generate_fib(Current, Limit) :-
    Current > Limit, nl.

% Interactive execution predicate
display_fibonacci :-
    write('Enter limit of terms (N): '),
    read(N),
    write('Fibonacci Sequence: '),
    generate_fib(0, N),
    write('-----------------------------'), nl,
    write('Name: [Prasanna Pokharel] | Roll: [24] | Lab: Lab II-7'), nl.
```

Output:

```
77 ?- display_fibonacci.
Enter limit of terms (N): 6.
Fibonacci Sequence: 0 1 1 2 3 5 8
------------------------------
Name: [Prasanna Pokharel] | Roll: [24] | Lab: Lab II-7
true .
```

8. Represent following facts using prolog:

   a. If a student studies they willl pass the exam.
   pass(X):- studied(X).

   b. If the student pass their exam they will be happy.
   happy(X):- pass(X).

   c. Radha studied for the exam.
   studied(radha).

   d. Rakesh studied for the exam.

   studied(rakesh).

   e. Anish studied for the exam.
   studied(anish).

   f. Rekha did not study for her exam.
   did_not_study(rekha).

   g. Bibek did not study for the exam
   did_not_study(bibek).

Using the facts, answer following :

   a. Did Anish pass? Yes.

```
80 ?- pass(anish).
true.

81 ?- print_footer.

------------------------------
Name: [Prasanna Pokharel] | Roll: [24] | Lab: Lab II-8
true.
```

b. List all the students that passed.

```
82 ?- pass(X).
X = radha ;
X = rakesh ;
X = anish.
```

c. Did rekha study? No.

```
89 ?- studied(rekha).
false.

90 ?- print_footer.

------------------------------
Name: [Prasanna Pokharel] | Roll: [24] | Lab: Lab II-8
true.
```

d. List all the students that did not study.

```
93 ?- did_not_study(X).
X = rekha ;
X = bibek .

93 ?- print_footer.

------------------------------
Name: [Prasanna Pokharel] | Roll: [24] | Lab: Lab II-8
true.
```

LAB III:

1. WAP to develop a sample expert system that performs disease diagnosis on the basis of the symptoms provided.

```
# Simple Expert System for Disease Diagnosis (COVID-19, Malaria, Food Poisoning)

print("Disease Diagnosis Expert System")

fever = input("Do you have fever? y/n ").strip().lower()
loss_of_taste = input("Do you have loss of taste or smell? y/n ").strip().lower()
chills = input("Do you have chills or shivering? y/n ").strip().lower()
vomiting = input("Do you have nausea or vomiting?y/n ").strip().lower()

if fever == "y" and loss_of_taste == "y":
    print("\nDiagnosis: You may have COVID-19.")
elif fever == "y" and chills == "y":
    print("\nDiagnosis: You may have Malaria.")
elif vomiting == "y" and fever == "n":
    print("\nDiagnosis: You may have Food Poisoning.")
else:
    print("\nDiagnosis: Disease not identified. Consult a doctor.")

print("Name: Prasanna Pokharel")
print("Rollno: 24")
print("LAB III-1")
```

OUTPUT:

```
Disease Diagnosis Expert System

Do you have fever? y/n y
Do you have loss of taste or smell? y/n n
Do you have chills or shivering? y/n y
Do you have nausea or vomiting? y/n y

Diagnosis: You may have Malaria.
Name: Prasanna Pokharel
Rollno: 24
LAB III-1
```

2. WAP to demonstrate working of Naïve Bayesian Classifier.

```
# Naive Bayes Classifier for Loan Approval
from sklearn.naive_bayes import GaussianNB

# Training data: [Monthly Income (in thousands), Credit Score]
X = [
    [25, 550],
    [30, 580],
    [35, 600],
    [60, 720],
    [75, 750],
    [90, 800]
]

# Output labels: Rejected or Approved
y = ["Rejected", "Rejected", "Rejected", "Approved", "Approved", "Approved"]

# Training the classifier
model = GaussianNB()
model.fit(X, y)

# User input
income = int(input("Enter monthly income (in thousands, e.g. 50): "))
credit_score = int(input("Enter credit score (300-850): "))

# Prediction
result = model.predict([[income, credit_score]])

print("\nLoan Status:", result[0])

# Student Detail
print("\nStudent Detail")
print("Name: Prasanna Pokharel")
print("Rollno: 24")
print("LAB III-2")
```

```
Enter monthly income (in thousands, e.g. 50): 76
Enter credit score (300-850): 730

Loan Status: Approved

Student Detail
Name: Prasanna Pokharel
Rollno: 24
LAB III-2
```

LAB-IV

1. Write a program to implement perceptron learning algorithm for AND, OR and NOT gate.

```
import numpy as np

class Perceptron:
    def __init__(self, input_size, lr=0.1, epochs=20):
        self.weights = np.zeros(input_size + 1)  # +1 for bias
        self.lr = lr
        self.epochs = epochs

    def step_function(self, x):
        return 1 if x >= 0 else 0

    def predict(self, x):
        # dot product with inputs + bias term
        z = np.dot(x, self.weights[1:]) + self.weights[0]
        return self.step_function(z)

    def train(self, X, y):
        for _ in range(self.epochs):
            for xi, target in zip(X, y):
                output = self.predict(xi)
                error = target - output
                self.weights[1:] += self.lr * error * xi
                self.weights[0] += self.lr * error  # bias update

def evaluate_gate(name, X, y):
    p = Perceptron(input_size=X.shape[1])
    p.train(X, y)
    print(f"--- {name} Gate ---")
    for xi, target in zip(X, y):
        pred = p.predict(xi)
        print(f"Input: {xi} | Predicted: {pred} | Actual: {target}")
    print(f"Weights: {p.weights[1:]}, Bias: {p.weights[0]}\n")

# Datasets
X_2input = np.array([[0, 0], [0, 1], [1, 0], [1, 1]])
y_and = np.array([0, 0, 0, 1])
y_or  = np.array([0, 1, 1, 1])

X_1input = np.array([[0], [1]])
y_not = np.array([1, 0])

evaluate_gate("AND", X_2input, y_and)
evaluate_gate("OR", X_2input, y_or)
evaluate_gate("NOT", X_1input, y_not)
```

```
print("Name: Prasanna Pokharel")
print("Rollno: 24")
print("LAB IV-1")
```

OUTPUT:

```
--- AND Gate ---
Input: [0 0] | Predicted: 0 | Actual: 0
Input: [0 1] | Predicted: 0 | Actual: 0
Input: [1 0] | Predicted: 0 | Actual: 0
Input: [1 1] | Predicted: 1 | Actual: 1
Weights: [0.2 0.1], Bias: -0.20000000000000004

--- OR Gate ---
Input: [0 0] | Predicted: 0 | Actual: 0
Input: [0 1] | Predicted: 1 | Actual: 1
Input: [1 0] | Predicted: 1 | Actual: 1
Input: [1 1] | Predicted: 1 | Actual: 1
Weights: [0.1 0.1], Bias: -0.1

--- NOT Gate ---
Input: [0] | Predicted: 1 | Actual: 1
Input: [1] | Predicted: 0 | Actual: 0
Weights: [-0.1], Bias: 0.0

Name: Prasanna Pokharel
Rollno: 24
LAB IV-1
```

2. Write a program to implement back propagation algorithm for XOR gate.

```
import numpy as np

def sigmoid(x):
    return 1.0 / (1.0 + np.exp(-x))

def sigmoid_derivative(x):
    return x * (1.0 - x)

# XOR Input and Target
X = np.array([[0, 0], [0, 1], [1, 0], [1, 1]])
y = np.array([[0], [1], [1], [0]])

# Network Architecture: 2 inputs -> 2 hidden neurons -> 1 output neuron
np.random.seed(42)
input_dim, hidden_dim, output_dim = 2, 2, 1
lr = 0.5
epochs = 10000

# Initialize weights and biases
```

```
W_h = np.random.uniform(size=(input_dim, hidden_dim))
b_h = np.random.uniform(size=(1, hidden_dim))
W_o = np.random.uniform(size=(hidden_dim, output_dim))
b_o = np.random.uniform(size=(1, output_dim))

# Training Loop
for epoch in range(epochs):
    # Forward Pass
    hidden_input = np.dot(X, W_h) + b_h
    hidden_output = sigmoid(hidden_input)

    final_input = np.dot(hidden_output, W_o) + b_o
    final_output = sigmoid(final_input)

    # Backpropagation
    error = y - final_output
    d_output = error * sigmoid_derivative(final_output)

    hidden_error = d_output.dot(W_o.T)
    d_hidden = hidden_error * sigmoid_derivative(hidden_output)

    # Gradient Updates
    W_o += hidden_output.T.dot(d_output) * lr
    b_o += np.sum(d_output, axis=0, keepdims=True) * lr
    W_h += X.T.dot(d_hidden) * lr
    b_h += np.sum(d_hidden, axis=0, keepdims=True) * lr

print("--- XOR Gate (Backpropagation) Predictions ---")
for xi, target, pred in zip(X, y, final_output):
    print(f"Input: {xi} -> Raw Output: {pred[0]:.4f} -> Class: {round(pred[0])}
(Target: {target[0]})")

print("Name: Prasanna Pokharel")
print("Rollno: 24")
print("LAB IV-2")
```

Output:

```
--- XOR Gate (Backpropagation) Predictions ---
Input: [0 0] -> Raw Output: 0.0189 -> Class: 0 (Target: 0)
Input: [0 1] -> Raw Output: 0.9837 -> Class: 1 (Target: 1)
Input: [1 0] -> Raw Output: 0.9837 -> Class: 1 (Target: 1)
Input: [1 1] -> Raw Output: 0.0169 -> Class: 0 (Target: 0)
Name: Prasanna Pokharel
Rollno: 24
LAB IV-2
```

3. Write a program to solve Water Jug Problem.

(Problem statement: Given two jugs, a 4-gallon and 3-gallon having no measuring markers on them. There is a pump that can be used to fill the jugs with water and the water can be poured on the ground. How can you get exactly 2 gallons of water into 4- gallon jug?)

```
from collections import deque

def water_jug_bfs(cap_a=4, cap_b=3, target=2):
    # State: (jug_a, jug_b)
    initial_state = (0, 0)
    queue = deque([(initial_state, [])])
    visited = set([initial_state])

    while queue:
        (a, b), path = queue.popleft()

        # Goal condition: exactly target gallons in jug A
        if a == target:
            return path + [(a, b)]

        # Generate all 6 possible production rules
        possible_moves = [
            ((cap_a, b), f"Fill 4-gallon jug: ({cap_a}, {b})"),
            ((a, cap_b), f"Fill 3-gallon jug: ({a}, {cap_b})"),
            ((0, b), f"Empty 4-gallon jug: (0, {b})"),
            ((a, 0), f"Empty 3-gallon jug: ({a}, 0)"),
            # Pour A -> B
            ((max(0, a - (cap_b - b)), min(cap_b, b + a)),
             f"Pour from 4-gal to 3-gal: ({max(0, a - (cap_b - b))}, {min(cap_b, b
+ a)})"),
            # Pour B -> A
            ((min(cap_a, a + b), max(0, b - (cap_a - a))),
             f"Pour from 3-gal to 4-gal: ({min(cap_a, a + b)}, {max(0, b - (cap_a -
a))})")
        ]

        for next_state, action in possible_moves:
            if next_state not in visited:
                visited.add(next_state)
                queue.append((next_state, path + [(a, b, action)]))

    return None

solution = water_jug_bfs()
print("--- Water Jug Solution Steps ---")
for step, (a, b, action) in enumerate(solution[:-1], 1):
    print(f"Step {step}: {action}")
print(f"Goal Reached: State is {solution[-1]}\n")
```

```
print("Name: Prasanna Pokharel")
print("Rollno: 24")
print("LAB IV-3")
```

OUTPUT:

```
--- Water Jug Solution Steps ---
Step 1: Fill 4-gallon jug: (4, 0)
Step 2: Pour from 4-gal to 3-gal: (1, 3)
Step 3: Empty 3-gallon jug: (1, 0)
Step 4: Pour from 4-gal to 3-gal: (0, 1)
Step 5: Fill 4-gallon jug: (4, 1)
Step 6: Pour from 4-gal to 3-gal: (2, 3)
Goal Reached: State is (2, 3)

Name: Prasanna Pokharel
Rollno: 24
LAB IV-3
```

4. Write a program to demonstrate the steps in genetic algorithm taking suitable example problem.

```
import random

# Use Case: Find the optimal 4-bit combination '1111' (Decimal 15)
TARGET_FITNESS = 4

# Fitness function: Count of 1-bits in a 4-bit integer
def fitness(x):
    return bin(x).count('1')

# Initial population: 6 random 4-bit integers (0 to 15)
population = [random.randint(0, 15) for _ in range(6)]

generation = 0

while True:
    print(f"\nGeneration {generation}")
    print("Population:", [f"{x:04b}" for x in population])

    # Rank candidates by fitness
    scores = [(fitness(x), x) for x in population]
    scores.sort(reverse=True)

    parent1 = scores[0][1]
    parent2 = scores[1][1]
```

```
    print(f"Parents: {parent1:04b} (Fit: {scores[0][0]}), {parent2:04b} (Fit:
{scores[1][0]})")

    # Natural termination: stop as soon as perfect fitness (4) is reached
    if scores[0][0] == TARGET_FITNESS:
        best = parent1
        break

    # Crossover: split 4 bits at midpoint (top 2 bits: 12 / '1100', bottom 2 bits:
3 / '0011')
    child1 = (parent1 & 12) | (parent2 & 3)
    child2 = (parent2 & 12) | (parent1 & 3)

    # Mutation: 30% chance to flip the least significant bit
    if random.random() < 0.3:
        child1 ^= 1
    if random.random() < 0.3:
        child2 ^= 1

    # Next generation: 2 parents + 2 children + 2 new random candidates
    population = [
        parent1,
        parent2,
        child1,
        child2,
        random.randint(0, 15),
        random.randint(0, 15)
    ]

    generation += 1

print("\nOptimal Solution Found!")
print("Best Solution (Binary)  =", f"{best:04b}")
print("Best Solution (Decimal) =", best)
print(f"Fitness                  = {fitness(best)}/{TARGET_FITNESS}")
print("Generations Taken       =", generation)


print("\nPrasanna Pokharel")
print("Rollno: 24")
print("LAB IV-4")
```

OUTPUT:

```
Generation 0
Population: ['1000', '0001', '0101', '1000', '0101', '1100']
Parents: 1100 (Fit: 2), 0101 (Fit: 2)

Generation 1
Population: ['1100', '0101', '1101', '0100', '0100', '0111']
Parents: 1101 (Fit: 3), 0111 (Fit: 3)

Generation 2
Population: ['1101', '0111', '1111', '0100', '1100', '1000']
Parents: 1111 (Fit: 4), 1101 (Fit: 3)

Optimal Solution Found!
Best Solution (Binary)  = 1111
Best Solution (Decimal) = 15
Fitness                 = 4/4
Generations Taken       = 2

Prasanna Pokharel
Rollno: 24
LAB IV-4
```

5. WAP to demonstrate some NLP tasks using NLTK.

[Sentence tokenization, Word tokenization, stop words filtering, word stemming, POS tagging, etc.]

```python
import nltk
from nltk.tokenize import sent_tokenize, word_tokenize
from nltk.corpus import stopwords
from nltk.stem import PorterStemmer

# Download required resources (runs once)
nltk.download('punkt', quiet=True)
nltk.download('punkt_tab', quiet=True)
nltk.download('stopwords', quiet=True)
nltk.download('averaged_perceptron_tagger', quiet=True)
nltk.download('averaged_perceptron_tagger_eng', quiet=True)

sample_text = """Artificial Intelligence is rapidly transforming modern industries.
Engineers are building systems that can learn, reason, and adapt autonomously.
Does this technology simplify our daily challenges?"""

# 1. Sentence Tokenization
sentences = sent_tokenize(sample_text)
print("--- 1. Sentence Tokenization ---")
for idx, s in enumerate(sentences, 1):
    print(f"[{idx}] {s}")
```

```python
# 2. Word Tokenization
words = word_tokenize(sample_text)
print("\n--- 2. Word Tokenization (First 15 Words) ---")
print(words[:15])

# 3. Stopwords Filtering
stop_words = set(stopwords.words('english'))
filtered_words = [w for w in words if w.isalnum() and w.lower() not in stop_words]
print("\n--- 3. Stopwords Filtering (Sample) ---")
print(filtered_words[:12])

# 4. Stemming
stemmer = PorterStemmer()
stemmed_words = [stemmer.stem(w) for w in filtered_words]
print("\n--- 4. Porter Stemming ---")
for orig, stem in list(zip(filtered_words, stemmed_words))[:8]:
    print(f"{orig:<15} -> {stem}")

# 5. POS Tagging
pos_tags = nltk.pos_tag(word_tokenize(sentences[0]))
print("\n--- 5. Part-of-Speech (POS) Tagging (Sentence 1) ---")
for word, tag in pos_tags:
    print(f"{word:<15} : {tag}")

print("\nPrasanna Pokharel")
print("Rollno: 24")
print("LAB IV-5")
```

OUTPUT:

```
--- 1. Sentence Tokenization ---
[1] Artificial Intelligence is rapidly transforming modern industries.
[2] Engineers are building systems that can learn, reason, and adapt autonomously.
[3] Does this technology simplify our daily challenges?

--- 2. Word Tokenization (First 15 Words) ---
['Artificial', 'Intelligence', 'is', 'rapidly', 'transforming', 'modern', 'industries', '.', 'Engineers', 'are', 'building', 'systems', 'that', 'can', 'learn']

--- 3. Stopwords Filtering (Sample) ---
['Artificial', 'Intelligence', 'rapidly', 'transforming', 'modern', 'industries', 'Engineers', 'building', 'systems', 'learn', 'reason', 'adapt']

--- 4. Porter Stemming ---
Artificial      -> artifici
Intelligence    -> intellig
rapidly         -> rapidli
transforming    -> transform
modern          -> modern
industries      -> industri
Engineers       -> engin
building        -> build

--- 5. Part-of-Speech (POS) Tagging (Sentence 1) ---
Artificial      : JJ
Intelligence    : NNP
is              : VBZ
rapidly         : RB
transforming    : VBG
modern          : JJ
industries      : NNS
.               : .

Prasanna Pokharel
Rollno: 24
LAB IV-5
```